

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 2
Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

1. Sample Images

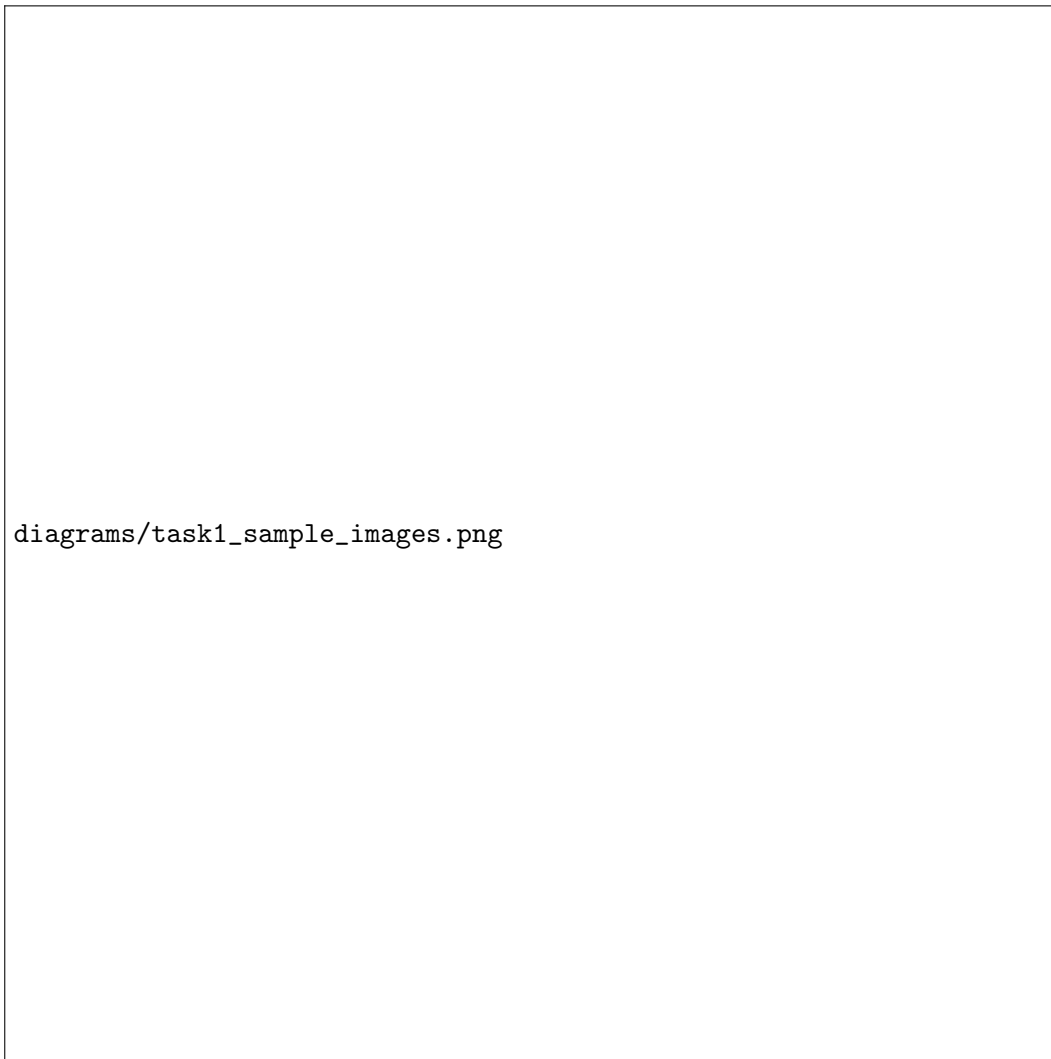


Figure 1: Sample Images from the Fashion-MNIST dataset

- All ten classes look visually distinct enough to tell apart by eye, but a few pairs like Shirt, Pullover and Coat share a very similar silhouette, which hints that the model might mix these up later.

2. Class Distribution

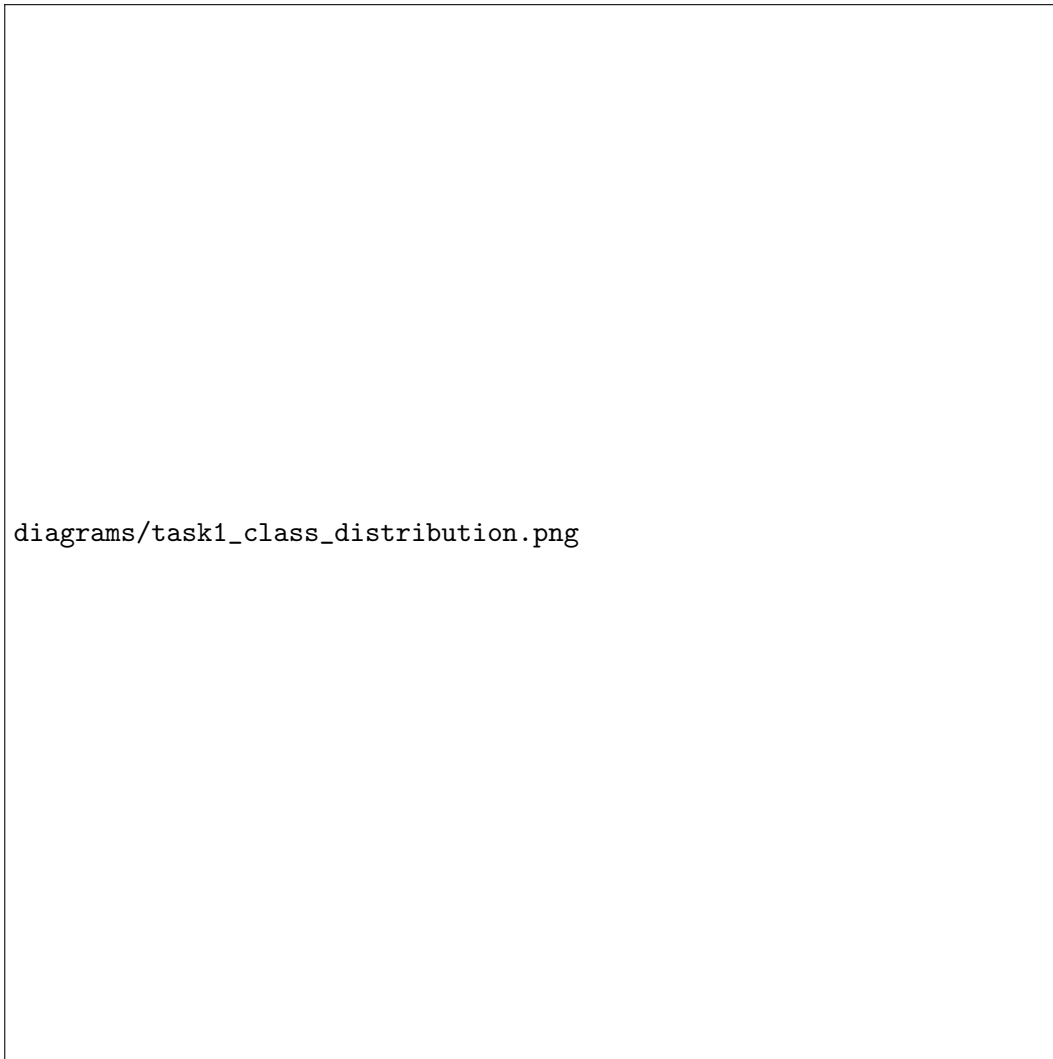


Figure 2: Training Dataset Class Distribution

- The training set is perfectly balanced with exactly 6,000 images per class (60,000 images across 10 classes), so there was no need for any class-imbalance handling like oversampling or class weights.

3. Training Accuracy vs Epoch



Figure 3: Training Accuracy versus Epoch

- Training accuracy rose steadily from about 82% in epoch 1 to 93.56% by epoch 20, without any sudden jumps, which shows the network was learning smoothly and the learning rate used with Adam was well suited for this problem.

4. Validation Accuracy vs Epoch



Figure 4: Validation Accuracy versus Epoch

- Validation accuracy improved quickly in the first few epochs and then flattened out around 88–89%, ending at 89.39%, roughly 4 points below training accuracy, which is a sign of mild overfitting setting in during later epochs.

5. Training Loss vs Epoch



Figure 5: Training Loss versus Epoch

- Training loss dropped consistently across all 20 epochs, ending at 0.1727, confirming that the optimizer kept finding improvements on the training data right until the end.

6. Validation Loss vs Epoch



Figure 6: Validation Loss versus Epoch

- Unlike training loss, validation loss stopped improving after roughly epoch 10 and even spiked briefly around epoch 18 (0.3694) before settling near 0.3466 at the end, which is the clearest evidence in this experiment that the baseline model was starting to overfit.

7. Confusion Matrix



Figure 7: Confusion Matrix of the Baseline MLP Model

- Almost all of the model’s mistakes are concentrated among the "Shirt", "T-shirt/Top", "Pullover" and "Coat" classes (Shirt has the lowest recall at just 65.2%), which makes sense since these upper-body garments look very similar once flattened into a 784-length pixel vector and the model loses spatial information that a CNN would have kept.

8. Hyperparameter Search Results

(Figure to be added after successful completion of Randomized Search.)

- Out of the 20 randomly sampled configurations, most landed between 82–89% cross-validation accuracy, but three combinations collapsed to near-random performance (as low as 9.9%), and in every one of those cases the learning rate was 0.1 paired with the Adam optimizer, showing that this combination makes training unstable and diverge.

9. Best Model Accuracy Comparison

(Figure to be added after obtaining optimized model results.)

- The best configuration found by the search reached 89.12% mean cross-validation accuracy, slightly ahead of the baseline model's 88.61% test accuracy, though a fair side-by-side comparison will only be possible once the optimized model is retrained and evaluated on the actual test set (this retraining and testing step is still pending as of this report).

9. Results

For the baseline model, Task 1–5 of the main experiment were fully completed: the MLP (784 → Dense(128, ReLU) → Dense(64, ReLU) → Dense(10, Softmax)) was trained for 20 epochs with batch size 32 using the Adam optimizer, and it took 104.26 seconds to train. It reached a final training accuracy of 93.56% and a final validation accuracy of 89.39%, with a test accuracy of 88.61% on the full 10,000-image test set.

For the hyperparameter optimization stage, since the SciKeras wrapper kept throwing a `scikit-learn/scikeras` version conflict (the `__sklearn_tags__` error) on the Kaggle kernel, the search was instead carried out with a manual Randomized Search loop (20 randomly sampled configurations, each evaluated with 5-fold cross validation, giving 100 model fits in total). The full search took 7,375.59 seconds (about 2 hours), which already gives an idea of how long a full Grid Search over this space would take – the complete grid has $3 \times 4 \times 3 \times 3 \times 3 \times 3 \times 4 \times 3 = 11,664$ possible combinations, so evaluating all of them with 5-fold CV would need roughly 58,320 model fits, which at the same per-fit speed would take somewhere around 50 days. This is exactly why Randomized Search was the practical choice here.

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	89.12%
Testing Accuracy	Pending (optimized model retraining/evaluation not yet run)

Performance Comparison

Metric	Baseline	Optimized
Accuracy	88.61%	– (pending)
Precision	88.57%	– (pending)
Recall	88.61%	– (pending)
F1-score	88.52%	– (pending)
Training Time	104.26 s	– (pending)

Note: Retraining the optimized model on the full training set and evaluating it on the held-out test set is the additional task that still needs to be run and submitted along with this report. Only the 5-fold cross-validation accuracy (89.12%) is available for the optimized configuration at the time of writing.

10. Discussion

1. Which optimization method was used?

Randomized Search was used instead of Grid Search, since the lab manual recommends it for a search space this large. Because of a version conflict between `scikit-learn` and `scikeras` on Kaggle (the `RandomizedSearchCV + KerasClassifier` combination kept throwing an `AttributeError` for `__sklearn_tags__`), the search was implemented manually: 20 hyperparameter combinations were sampled at random from the search space, and each one was evaluated using 5-fold cross validation on the training data, so it behaves the same way as `RandomizedSearchCV` would, just without depending on the broken wrapper.

2. Which hyperparameters were selected?

The best combination found was 3 hidden layers of 128 neurons each, sigmoid activation, the Adam optimizer with a learning rate of 0.001, no dropout, a batch size of 16, trained for 30 epochs. This combination reached a mean cross-validation accuracy of 89.12%, the highest of all 20 sampled configurations.

3. Did optimization improve performance?

Based on cross-validation accuracy alone, yes – 89.12% for the optimized configuration versus 88.61% test accuracy for the baseline, so roughly half a percentage point higher. That said, this isn't a completely fair comparison yet since one number is a 5-fold CV average on the training data and the other is a single test-set accuracy. The optimized model still needs to be retrained on the full training set and evaluated on the actual test set to confirm the improvement holds up outside cross-validation.

4. Which hyperparameter had the greatest impact?

Learning rate, and specifically how it interacts with the optimizer. Every configuration that used a learning rate of 0.1 together with Adam collapsed to near-random accuracy (9.9%, 18.64% and 21.64% in three separate runs), which is a classic sign of the optimizer overshooting and diverging. Interestingly, a learning rate of 0.1 was completely fine when paired with SGD instead of Adam (accuracies of 87.69%, 88.08% and 88.73% in three other runs), which shows that Adam's adaptive updates simply cannot tolerate as large a step size as SGD can. So learning rate on its own isn't the whole story, but it's the hyperparameter that most easily broke training when set wrong.

5. Compare Grid Search and Randomized Search.

Grid Search checks every single combination in the search space, so it's guaranteed to find the best combination within the grid, but it doesn't scale well: this search space alone has 11,664 possible combinations, and with 5-fold CV that's 58,320 model fits – completely impractical to run on a single Kaggle session. Randomized Search instead samples a fixed number of combinations (20, in this case) and only needed 100 fits, finishing in about 2 hours. It doesn't guarantee finding the global best, but in practice it gets close enough, especially when the search space is this large, which is exactly why the lab manual recommends it here.

6. Recommend the best MLP configuration.

Based on the search results, the recommended configuration is: 3 hidden layers with 128 neurons each, sigmoid activation, Adam optimizer, learning rate 0.001, batch size 16, 30 epochs, and no dropout. This is deeper than the original baseline architecture (which only used 2 hidden layers of 128 and 64 neurons) and trains for longer (30 epochs instead of 20), which together seem to give it a slight edge in cross-validation accuracy. The final recommendation should be confirmed once this configuration is retrained and tested on the held-out test set.

11. Conclusion

This experiment covered the full pipeline for building and tuning an MLP on Fashion-MNIST, from loading and flattening the images through to automated hyperparameter search. The baseline MLP (2 hidden layers, 128 and 64 neurons, ReLU, Adam) reached a respectable 88.61% test accuracy, with the confusion matrix showing that almost all of its errors came from confusing visually similar upper-body

clothing classes like Shirt, Pullover and Coat – a limitation that comes from flattening the images and losing spatial structure, something a CNN would handle better.

For hyperparameter optimization, a manual Randomized Search (since `scikeras` wasn't compatible with the installed `scikit-learn` version) tested 20 combinations with 5-fold cross validation and found a deeper 3-hidden-layer, sigmoid-activation configuration that reached 89.12% CV accuracy, a modest improvement over the baseline. The search also made it very clear that learning rate, especially in combination with the optimizer, is the hyperparameter most likely to break training entirely if set too high.

The one piece of work still pending is the additional task that needs to be completed before final submission: retraining the best configuration on the full training set, evaluating it on the test set, and filling in the Optimized column of the performance comparison table (Section 9) along with the two remaining plots (hyperparameter search results and best model accuracy comparison). Once that is done, this report will have a complete, apples-to-apples comparison between the baseline and optimized MLP models.

12. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.